



**Università  
degli Studi  
di Palermo**

CORSO DI LAUREA IN INFORMATICA  
DIPARTIMENTO DI MATEMATICA E INFORMATICA

# **SISTEMI DI RACCOMANDAZIONE**

AUTORE

**Giampiero Fuschi**

DOCENTE TUTOR

**Simona Ester Rombo**

**ANNO ACCADEMICO 2024/2025**

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. Introduzione</b>                                              | <b>2</b>  |
| 1.1 Organizzazione del lavoro                                       | 2         |
| 1.2 Cosa sono i sistemi di raccomandazione                          | 2         |
| 1.3 Benefici dei sistemi di raccomandazione                         | 3         |
| 1.4 “Knowledge” (conoscenza) nei sistemi di raccomandazione         | 5         |
| <b>2. Categorie di Sistemi di Raccomandazione</b>                   | <b>6</b>  |
| 2.1 Popularity based recommendation                                 | 6         |
| 2.2 Content based recommendation                                    | 7         |
| 2.2.1 Knowledge based recommendation                                | 8         |
| 2.3 Collaborative recommendation                                    | 8         |
| 2.4 Hybrid recommendation systems                                   | 10        |
| 2.5 Valutazione dei sistemi di raccomandazione                      | 11        |
| <b>3. Vector Embedding</b>                                          | <b>12</b> |
| 3.1 Cosa è il vector embedding                                      | 12        |
| 3.2 Parentesi su vector embedding nei Deep Learning RS              | 12        |
| <b>4. Focus dell’analisi</b>                                        | <b>13</b> |
| 4.1 Introduzione alle tecniche di Matrix Factorization              | 13        |
| 4.2 Modello base di fattorizzazione                                 | 14        |
| 4.2.1 Stochastic Gradient Descent                                   | 16        |
| 4.2.2 Alternating Least Squares                                     | 16        |
| 4.3 Fonti esterne, dinamiche temporali e confidence                 | 16        |
| 4.4 Resilient Distributed Datasets                                  | 18        |
| 4.5 Spark Programming Interface                                     | 18        |
| 4.6 Rappresentazione dei RDD                                        | 19        |
| 4.7 Job Scheduling                                                  | 20        |
| <b>5. Implementazione RS collaborativo con Spark</b>                | <b>21</b> |
| 5.1 Dataset Scelto                                                  | 21        |
| 5.2 Definizione della classe rating                                 | 21        |
| 5.3 Fase di pre-processing                                          | 22        |
| 5.4 Fase di Training e Test                                         | 22        |
| 5.5 Struttura dell’output del modello                               | 24        |
| 5.6 Considerazioni sull’output del modello                          | 25        |
| 5.7 Considerazioni sul Root Mean Square Error e parametro $\lambda$ | 27        |
| <b>6. Conclusioni</b>                                               | <b>28</b> |
| <b>7. Bibliografia</b>                                              | <b>29</b> |

# 1. Introduzione

## 1.1 Organizzazione del lavoro

Questo lavoro si divide in due parti principali:

1. La prima parte, che va dal capitolo 1 al capitolo 4, riguarda la ricerca e l'analisi sul campo dei Sistemi di Raccomandazione: i loro benefici e le loro categorie, con un focus sulla tecnica di Matrix Factorization e sui Resilient Distributed Datasets.
2. La parte finale, che corrisponde al capitolo 5, riguarda invece l'attività di sviluppo di un Sistema di Raccomandazione collaborativo tramite Apache Spark, tenendo conto delle considerazioni fatte nella prima parte.

## 1.2 Cosa sono i sistemi di raccomandazione

I sistemi di raccomandazione sono uno strumento fondamentale che trova applicazione in numerosi settori, come l'e-commerce, i social network e lo streaming audio e video. In particolare, piattaforme come Amazon, Instagram, Netflix e Spotify fanno forte uso di svariate tecniche di machine learning, data mining, user modeling, case-based reasoning, e constraint satisfaction, per raccomandare agli utenti prodotti o contenuti che possano essere di loro interesse. Ad esempio, la homepage di YouTube si occupa di mostrare una lista di video consigliati all'utente sulla base di svariate metriche (che approfondiremo più avanti), la quale è frutto di un complesso sistema di raccomandazione. Un buon sistema di raccomandazione deve accertarsi che i video consigliati siano effettivamente di interesse per l'utente, così da aumentare l'engagement con la piattaforma.

Una mancanza di correlazione tra i consigli della piattaforma e gli interessi dell'utente si tradurrebbe in una distrazione percepita da parte dello stesso.

[1] Da un punto di vista formale, il recommendation system può essere definito come segue:  
Sia “C” l'insieme di tutti gli utenti e sia “S” l'insieme dei possibili elementi da consigliare.  
Sia “u” una funzione utile a misurare l'utilità di un oggetto “s” per un utente “c”, ovvero:

$$u: C \times S \Rightarrow R,$$

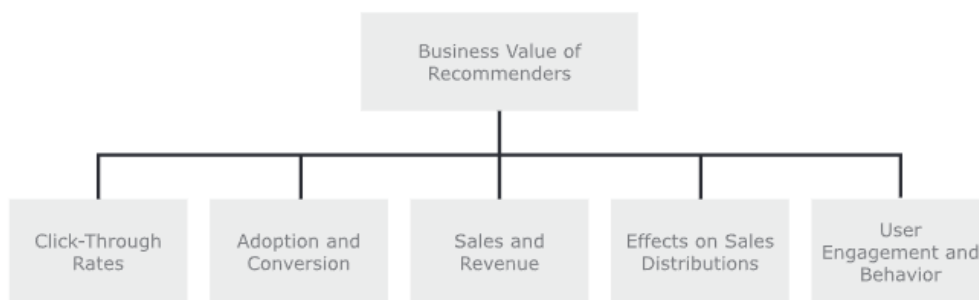
con “R” insieme totalmente ordinato (interi non negativi o numeri reali entro un certo range), allora per ogni utente “c ∈ C”, vogliamo scegliere l'elemento “s ∈ S” che massimizza la funzione “u” per quell'utente.

### 1.3 Benefici dei sistemi di raccomandazione

È chiaro che il beneficio principale di un buon sistema di raccomandazione per l'utente sia la fruizione di una piattaforma “fatta su misura” per lui.

Per l'azienda, invece, i Recommender Systems portano a un notevole aumento dei profitti: nel caso delle piattaforme di e-commerce, porteranno l'utente a visionare (e di conseguenza acquistare) più prodotti; mentre nel caso di una piattaforma di streaming, porterà l'utente ad interagire maggiormente con quest'ultima.

Alcuni studi [2] analizzano concretamente i reali benefici dei RS in termini di business value, andando a valutare le metriche illustrate in *Figura 1*:



[2] *Figura 1. Metriche per la misurazione del business value*

- **CTR (Click-Through Rates):** Quanti “click” vengono accumulati dalle raccomandazioni. Più click corrispondono ad un sistema di raccomandazione più efficace (usato principalmente nel campo delle news).
- **Adoption and Conversion:** Valuta se il click è sintomo di un interesse momentaneo o l'utente sta “accettando” il consiglio (adoption). Questa informazione viene tradotta in un obiettivo concreto di business (conversion). Ad esempio, su Youtube, i click sono contati solo se l'utente guarda una determinata porzione di video.
- **Sales and Revenue:** Valuta se il fattore adoption ha effettivamente prodotto un guadagno. Questo infatti non è necessariamente vero: un utente potrebbe avere acquistato un oggetto consigliato dal sistema di raccomandazione, ma che avrebbe comunque acquistato. A riguardo, percentuali precise non sono disponibili, ma si registrano incrementi di profitto che arrivano fino al 50% in base all'approccio seguito e al tipo di piattaforma che ne fa uso.
- **Effects on Sales Distributions:** Misura non solo quanti più prodotti gli utenti comprano grazie ai RS, ma anche la varietà degli stessi (la distribuzione), in base all'efficacia dei consigli.

- **User Engagement and Behaviour:** Misura quanto l'utente interagisce con la piattaforma (la lunghezza delle sessioni, la durata degli abbonamenti e così via). In particolare, si parla di “user retention”.

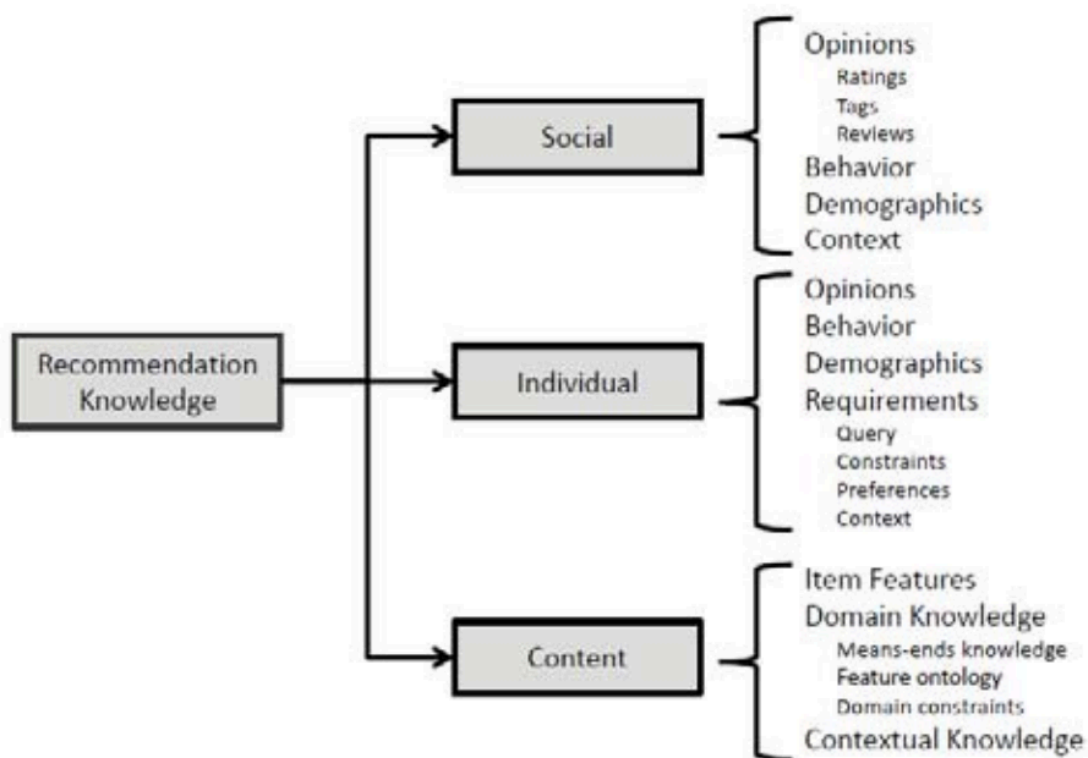
Chiaramente, anche il sistema di raccomandazione scelto va ad influire sull'efficacia in termini di business.

## 1.4 “Knowledge” (conoscenza) nei sistemi di raccomandazione

L’efficienza degli algoritmi di raccomandazione dipende inoltre fortemente dalla fonte di informazioni da cui attingono per le loro valutazioni.

In particolare, come approfondito in “Recommender Systems: An Overview” [1], sono presenti tre tipi di base di conoscenza, illustrati in *Figura 2*:

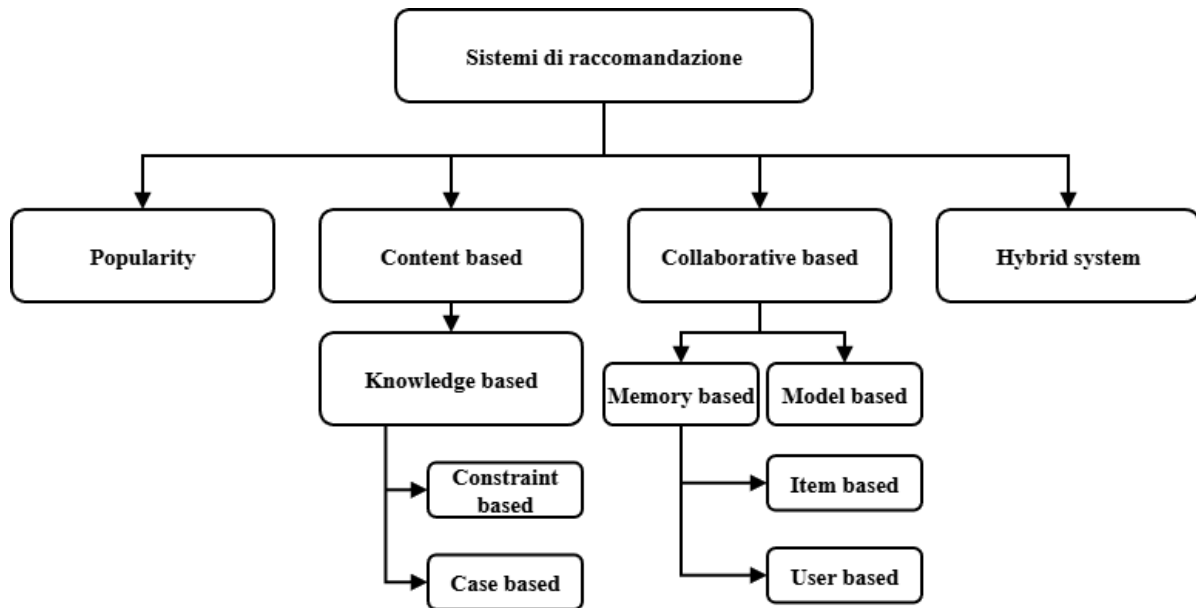
- ***Social knowledge*** (conoscenza sociale) dell’utenza generale del servizio offerto.
- ***Individual knowledge*** (conoscenza individuale) del singolo utente e dei suoi interessi.
- ***Content knowledge*** (conoscenza dei contenuti) riguardante gli oggetti



[1] *Figura 2. Categorizzazione della conoscenza nei Recommender Systems*

Il concetto di “conoscenza” (knowledge) è di fondamentale importanza per ogni sistema di raccomandazione, ma è particolarmente rilevante nei knowledge based recommender systems, che verrà descritto a breve.

## 2. Categorie di Sistemi di Raccomandazione



*Figura 3. Overview dei principali tipi di RS*

### 2.1 Popularity based recommendation

Uno dei modelli più semplici: si basa sull'idea che se un prodotto è popolare, allora questo può essere consigliato all'utente. La sua implementazione è piuttosto semplice, poiché non necessita di altro che contatori e/o query per il suo corretto funzionamento.

Per ovvi motivi, non tiene realmente conto delle preferenze dell'utente, dunque non massimizza la utility function. Può essere tuttavia combinato ai successivi sistemi di raccomandazione.

## 2.2 Content based recommendation

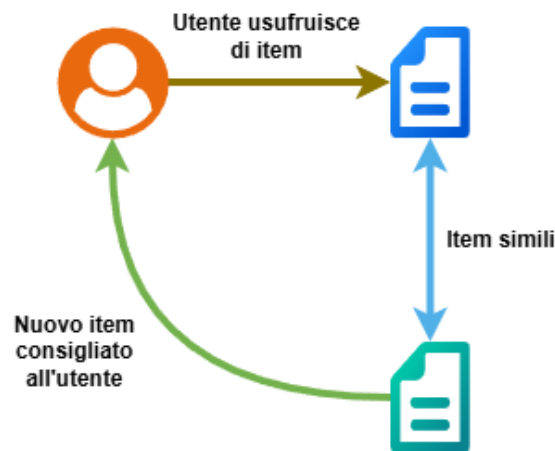


Figura 4. Content based Recommendation

Prima dell'arrivo della *collaborative recommendation*, la priorità era combinare unicamente la conoscenza sugli item alle preferenze degli utenti.

Questo modello è strettamente legato al machine learning supervisionato. In particolare, possiamo creare classificatori specifici per ogni utente, e composti da 2 classi: “*useful to user X*” e “*not useful to user X*”.

La qualità delle feature degli item è altresì importante, poiché è necessario che questi vengano descritti in modo tale da permettere un apprendimento significativo delle preferenze dell'utente. Un banale esempio di content based recommendation, consiste nel consigliare a un utente un item simile a uno che ha già acquistato precedentemente.

### 2.2.1 Knowledge based recommendation

[4] Si tratta di una sottocategoria dei sistemi content based, nella quale alle feature intrinseche degli item si aggiunge “conoscenza” (knowledge) normalmente non prevista in altri tipi di design: *user requirements* (requisiti dell’utente) e *domain knowledge* (conoscenza del dominio sul quale opera il settore). È ovviamente possibile trovare tali feature anche in altri modelli, come quello collaborativo, ma ciò che contraddistingue i knowledge based recommender system, è l’enfasi sulle necessità dell’utente e in che modo gli item consigliati possono soddisfare quelle necessità. Proprio per questo motivo, sono più difficili da caratterizzare rispetto agli altri modelli. I due principali approcci di questa sottocategoria sono:

- **Case based:** trattano la raccomandazione come caso di similarità. Ad esempio, trovare un prodotto simile che possa soddisfare l’utente, considerando che il concetto di “similitudine” dipende dalla conoscenza di dominio.
- **Constraint based:** tengono conto di restrizioni specifiche imposte dall’utente, ad esempio i filtri.

## 2.3 Collaborative recommendation



Figura 5. Collaborative based Recommendation

Si fonda sull’idea di una continuità di preferenze tra utenti. Se un utente “a” simile ad un utente “b”, mostra interesse nei confronti di un item “s”, allora quell’item può essere consigliato all’utente “b”. In alternativa, se due utenti hanno la stessa utility per gli item che vanno da indice 1 a indice k, c’è una buona probabilità che avranno la stessa utility per l’item k+1.

Il principale vantaggio del modello collaborativo è la semplicità, e il problema del calcolo della utility si trasforma nella necessità di estrapolare valori mancanti dalle *ratings matrix*, nelle quali ad ogni utente corrisponde una riga, e ad ogni item una colonna.

Per individuare utenti simili tra loro si utilizzavano in precedenza operazioni di nearest neighbour (memory based) sui dati grezzi, ma adesso si prediligono tecniche di riduzione della dimensionalità e fattorizzazione delle matrici (oggetto d'analisi), le quali permettono di proiettare gli utenti (e se necessario anche gli item) in uno spazio compatto, generando quelli che più avanti spiegheremo essere vector embeddings.

Si presentano tre principali problemi nel modello collaborativo:

- **Cold start:** i nuovi item sono difficili da consigliare agli utenti senza informazioni aggiuntive, e i nuovi utenti riceveranno raccomandazioni più “rumorose”.
- **Distribuzione dei rating:** potrebbero esserci item considerati “blockbuster”, che influenzano negativamente la distribuzione dei rating a scapito di numerosi item che invece risultano carenti di rating.
- **Sybil profiles:** utenti maliziosi potrebbero creare profili pseudonimi per influenzare i rating degli item.

## 2.4 Hybrid recommendation systems

L'utilizzo dei modelli ibridi è senza dubbio il più diffuso nelle piattaforme moderne, con lo scopo di superare le problematiche legate ai diversi sistemi di raccomandazione, migliorando le prestazioni del sistema. La *Tabella 1* illustra alcuni dei modelli ibridi e le loro caratteristiche:

| Hybrid Method                  | Descrizione                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Weighted Hybridization</b>  | È una tecnica in cui il peso viene modificato progressivamente in base al grado di accordo tra la valutazione dell'utente su un articolo e la valutazione prevista dal sistema di raccomandazione.                                                                                                                                                      |
| <b>Switching Hybridization</b> | Una strategia che prevede il cambio del modello di raccomandazione applicato in base alle circostanze o al contesto specifico.                                                                                                                                                                                                                          |
| <b>Cascaded Hybridization</b>  | Dopo aver utilizzato un modello per generare un insieme di candidati affini alle preferenze dell'utente, l'approccio combina il modello precedente con un altro per ordinare i candidati secondo le preferenze dell'utente.                                                                                                                             |
| <b>Mixed Hybridization</b>     | Quando vengono effettuate molte raccomandazioni simultanee, il Content-Based Filtering suggerisce prodotti basandosi sulle descrizioni senza richiedere valutazioni. Per risolvere il problema del "cold start" (nuovi articoli senza info), questo approccio integra i suggerimenti usando i dati storici dell'utente raccolti all'avvio del servizio. |
| <b>Feature-Combination</b>     | Strategia in cui viene utilizzato il Collaborative Filtering per i dati in evidenza e i dati di esempio degli articoli, mentre viene utilizzato un modello Content-Based per i dati aumentati.                                                                                                                                                          |
| <b>Feature-Augmentation</b>    | Un approccio ibrido in cui un modello di raccomandazione viene utilizzato per classificare il punteggio di preferenza o un attributo di un articolo, e le informazioni ottenute vengono poi inserite nel modello di raccomandazione successivo.                                                                                                         |
| <b>Meta-Level</b>              | Tecnica che consiste nell'usare l'intero modello di un sistema di raccomandazione come dato di input per alimentare il modello di un altro sistema. Le preferenze dell'utente vengono condensate tramite il Meta-Level, facilitando il meccanismo collaborativo rispetto all'uso di dati di valutazione grezzi.                                         |

[3] *Tabella 1. Categorie di Hybrid Recommendation*

## 2.5 Valutazione dei sistemi di raccomandazione

L'efficacia dei sistemi di raccomandazione viene valutata secondo due metriche principali:

- **Accuratezza delle previsioni dei rating:** dati i rating predetti dal sistema, si verifica quanti di essi corrispondono a quelli noti ma nascosti al sistema stesso.
- **Precisione delle liste di raccomandazione:** data una lista di elementi raccomandati dal sistema, si verifica quanti di questi corrispondono ad elementi che sono realmente apprezzati dall'utente.

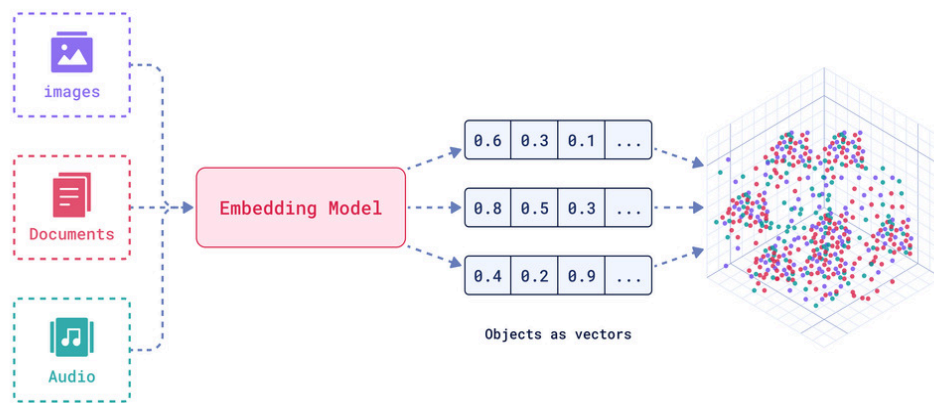
## 3. Vector Embedding

### 3.1 Cosa è il vector embedding

L'idea alla base del vector embedding è quella di trasformare un oggetto discreto in un punto geometrico nello spazio. In questo modo, generando il vector embedding di un elemento, sarà possibile trovare un elemento simile ragionando per vicinanza dei due punti nello spazio.

[5] In particolare, ad ogni feature viene associato un embedding unico  $E \in \mathbb{R}^{n \times d}$ , dove  $n$  corrisponde al numero di feature e  $d$  alla dimensione del vector embedding.

Naturalmente, nel nostro caso, per feature si intendono item e utenti.



[8] *Figura 6. Vector embedding*

Come anticipato in precedenza, gli embedding sono fortemente utilizzati nei Collaborative recommendation system, per individuare utenti simili, ma anche nei content based recommendation system, per individuare item simili tra loro.

Naturalmente, il concetto di embedding può essere esteso a numerosi tipi di dato, come testo, immagini, video e così via.

### 3.2 Parentesi su vector embedding nei Deep Learning RS

Per quanto il concetto di embedding possa essere implementato tramite tecniche di fattorizzazione matriciale che saranno oggetto di approfondimento, esso è anche alla base dei DLRS (Deep Learning Recommendation Systems), dove è la rete neurale ad occuparsi di processare e analizzare gli embedding delle feature, e di interagire con essi.

## 4. Focus dell'analisi

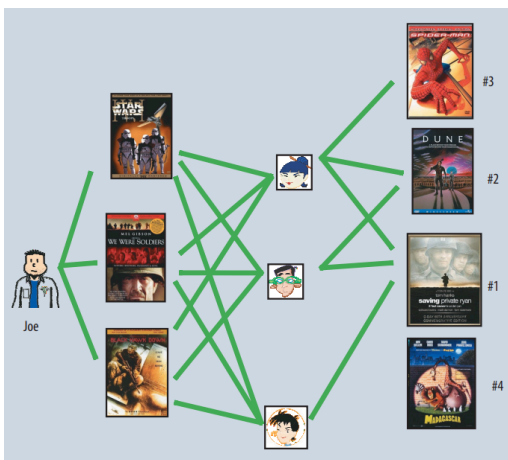
### 4.1 Introduzione alle tecniche di Matrix Factorization

Come anticipato, le tecniche fattorizzazione di matrici (*latent factor models*) sono preferibili alle semplici operazioni di nearest neighbour (*neighborhood methods*).

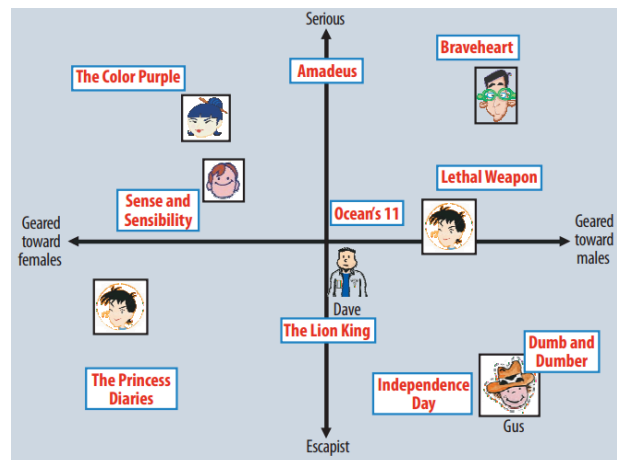
In queste ultime, si è interessati alle relazioni tra items, o alternativamente, tra utenti. Per esempio, due items sono considerati simili se i ratings nella matrice di valutazione di un dato utente sono vicini.

Come spiegato in “Matrix Factorization Techniques for Recommender Systems” [6] i *latent factor models* forniscono invece un approccio alternativo che caratterizza items e users tramite fattori multipli dedotti da pattern legati ai rating (ad esempio, nel caso dei film, questi pattern potrebbero essere più o meno ovvi, come il genere del film o la profondità della caratterizzazione dei personaggi).

Nello studio citato, la differenza tra i due metodi viene presentata come di seguito.



[6] Figura 7a. Neighborhood methods



[6] Figura 7b. Latent factor models

La fattorizzazione di matrici viene dunque definita come tecnica per la realizzazione dei latent factor models: nella sua forma più semplice, un'alta corrispondenza tra item e user factors porta ad una raccomandazione.

Le matrici vengono organizzate in modo che una dimensione rappresenti gli user e l'altra gli item di interesse. Il rating ritenuto più conveniente è quello fornito tramite feedback esplicito direttamente dall'utente (like o dislike, stelle e votazioni), e considerato che ad ogni cella corrisponderà il rating che l'utente  $i$  ha assegnato all'item  $j$  (un film o una serie TV nel caso

di Netflix), otterremo una matrice sparsa (perchè un utente valuterà solo una piccola percentuale di tutti gli item disponibili).

Se il feedback esplicito non è disponibile, la fattorizzazione di matrici ci permette di dedurre un feedback implicito osservando il comportamento dell'utente. Utilizzando il feedback implicito, avremo a disposizione una matrice densa.

## 4.2 Modello base di fattorizzazione

Nei Matrix factorization models [6], utenti e oggetti sono memorizzati in spazi latenti congiunti di dimensione  $f$ , in modo che interazioni user-item possano essere modellate come prodotti vettoriali in quello spazio.

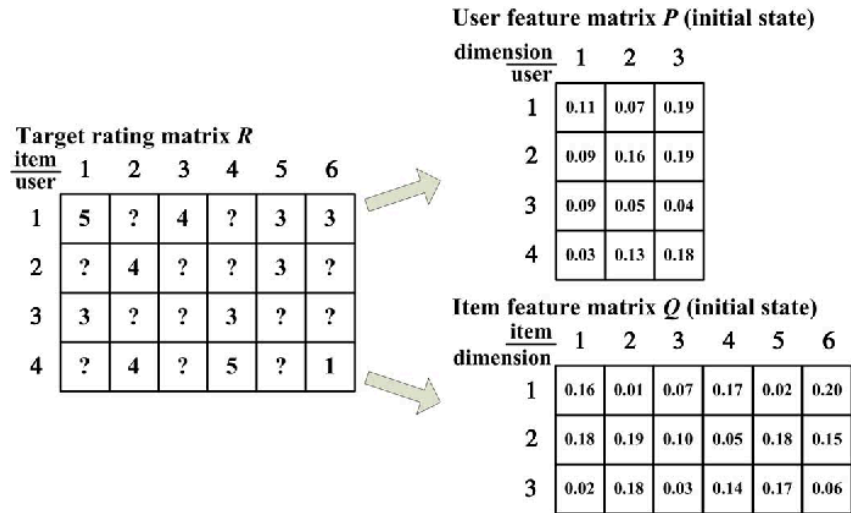
In parole povere, ogni item è rappresentato da un vettore  $p_i$  di dimensione  $f$ , ed ogni utente è anch'esso rappresentato da un vettore  $q_u$  di dimensione  $f$ .

In questo modo, per ogni item  $i$ , ogni elemento del vettore  $p_i$  (che può essere positivo o negativo) misura quanto l'item “possiede” la caratteristica legata al rispettivo indice (se al terzo elemento del vettore corrisponde il fattore “commedia” e il film è un film horror, il valore sarà negativo).

Analogamente, per ogni utente  $u$ , ogni elemento del vettore  $p_u$  (che può essere positivo o negativo) misura quanto l'utente è interessato alla caratteristica legata al rispettivo indice (se al quinto elemento del vettore corrisponde il fattore “horror” e l'utente ama i film horror, il valore sarà positivo).

Il prodotto tra vettore item trasposto e vettore user mostra chiaramente il rapporto tra uno specifico utente e uno specifico item, e approssima il rating dell'utente rispetto a quell'item.

$$\hat{r}_{ui} = q_i^T p_u.$$



[6] Figura 8. Processo di Matrix Factorization

La sfida diventa dunque quella di mappare ogni oggetto ad ogni utente, in modo da permettere al sistema di effettuare predizioni su larga scala per nuovi item, utilizzando l'equazione precedente.

Tale modello è strettamente legato alla SVD (Singular Value Decomposition), che risulta indefinita se applicata al filtro collaborativo nella sua forma convenzionale.

L'algoritmo viene addestrato solo sui voti  $r_{ui}$  realmente osservati cercando di evitare l'overfitting utilizzando una tecnica che minimizza l'errore quadratico regolarizzato, rappresentato dalla seguente funzione di costo:

$$\min_{q^*, p^*} \sum_{(u,i) \in \kappa} (r_{ui} - q_i^T p_u)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

Qui  $\kappa$  è l'insieme delle coppie  $(u, i)$  per le quali conosciamo il rating  $r_{ui}$  (il nostro training set), mentre  $\lambda$  è la costante che decide quanto peso dare alla regolarizzazione (che riduce effettivamente l'overfitting) e che viene scelta tramite cross-validation.

Se  $\lambda$  è troppo piccola, verrà ignorata la regolarizzazione sommata all'errore reale (risultato del quadrato di rating reale - predizione) portando ad overfitting, mentre se è troppo grande si darà priorità a vettori troppo piccoli.

Minimizzare la funzione di costo significa dunque trovare il punto di equilibrio tra  $\lambda$ ,  $p_i$  e  $q_u$ , ed esistono due principali metodi.

### 4.2.1 Stochastic Gradient Descent

Per ogni caso di addestramento, il sistema calcola l'errore di predizione :

$$e_{ui} \stackrel{def}{=} r_{ui} - q_i^T p_u.$$

Dopodichè modifica i parametri di una grandezza proporzionale a  $\gamma$  nella direzione opposta al gradiente:

- $q_i \leftarrow q_i + \gamma \cdot (e_{ui} \cdot p_u - \lambda \cdot q_i)$
- $p_u \leftarrow p_u + \gamma \cdot (e_{ui} \cdot q_i - \lambda \cdot p_u)$

### 4.2.2 Alternating Least Squares

Le tecniche di ALS funzionano alternando un fix delle componenti di  $p_i$  e un fix delle componenti di  $q_u$ .

Quando le componenti di  $p_i$  sono fissate, il sistema calcola le componenti di  $q_u$ , e viceversa, finché non si arriva ad una convergenza. ALS è particolarmente favorevole in due casi: se il sistema può utilizzare la parallelizzazione, e se il sistema si basa su dati impliciti.

Bisogna anche tenere conto del bias. In particolare, per un dato rating  $r_{ui}$ , il bias  $b_{ui}$  è:

$$b_{ui} = \mu + b_i + b_u$$

Dove  $\mu$  è il rating medio, e  $b_i$  e  $b_u$  indicano le deviazioni osservate sull'item  $i$  e l'utente  $u$ .

A questo punto la funzione da minimizzare diventa:

$$\min_{p^*, q^*, b^*} \sum_{(u,i) \in \mathcal{K}} (r_{ui} - \mu - b_u - b_i - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

## 4.3 Fonti esterne, dinamiche temporali e confidence

L'articolo [6] approfondisce inoltre le preferenze degli utenti, che possono essere dedotte non solo da feedback espliciti, ma anche tramite analisi comportamentale derivante da elementi dipendenti dal tipo di piattaforma, come acquisti precedenti, cronologia di ricerca, e anche attributi associati agli utenti come quelli demografici.

È inoltre possibile tenere conto del fatto che le preferenze generali (popolarità) e individuali possono variare con il passare del tempo. In questo caso, i parametri (compreso il bias) agiscono in funzione del tempo  $t$ .

$$\hat{r}_{ui}(t) = \mu + b_i(t) + b_u(t) + q_i^T p_u(t)$$

Il concetto di confidence  $c_{ui}$  nell'osservazione di un rating  $r_{ui}$  può inoltre essere tenuto in considerazione: quanto siamo sicuri dell'osservazione effettuata?

$$\min_{p^*, q^*, b^*} \sum_{(u,i) \in \mathcal{K}} c_{ui} (r_{ui} - \mu - b_u - b_i - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

## 4.4 Resilient Distributed Datasets

L'articolo esaminato [7] approfondisce i Resilient Distributed Datasets: si tratta di un'astrazione di memoria distribuita e parallela, che consente di effettuare computazioni “*in-memory*” (nella memoria centrale piuttosto che su disco rigido, per ottenere velocità notevolmente più alte), in maniera “*fault-tolerant*” (il sistema continua a operare senza interruzioni nonostante eventuali errori). L'obiettivo principale dei RDD, implementati nel sistema Spark, è migliorare l'efficienza di framework precedenti, algoritmi iterativi (come k-means e clustering nel campo del machine learning) e strumenti di data mining.

Formalmente, i RDD sono immutabili e possono essere creati unicamente tramite operazioni deterministiche su un sistema stabile di archiviazione (*stable storage*) o su un altro RDD, note come trasformazioni (ad esempio *map*, *filter*, e *join*). Queste trasformazioni sono “*coarse-grained*” (a grana grossa): sono applicate all'intero dataset o a una porzione di esso in un colpo solo, così da garantire “*fault-tolerance*” loggando le informazioni che hanno portato alla costruzione del dataset invece degli effettivi dati. In questo modo, ogni RDD ha abbastanza informazioni su come esso è stato derivato da altri RDD, e i dati possono essere ricostruiti senza doverli necessariamente replicare.

Naturalmente, poiché possono essere creati unicamente tramite trasformazioni a grana grossa, i RDD sono ristretti nella loro utilità ad applicazioni che effettuano bulk write.

## 4.5 Spark Programming Interface

Spark espone i RDD tramite API, dove ogni dataset è un oggetto e le trasformazioni vengono effettuate applicando metodi su questi oggetti. Il workflow prevede di definire uno o più RDD applicando trasformazioni su un sistema stabile di archiviazione, per poi utilizzare “azioni” (*actions*) come *count* (restituisce il numero di elementi del dataset), *collect* (restituisce gli elementi del dataset), e *save* (output degli elementi del dataset su uno storage). Il metodo *persist* serve invece ad indicare quali RDD verranno riutilizzati in operazioni future. Le trasformazioni vengono registrate nel lineage graph (grafo di ereditarietà) e vengono eseguite solo quando si esegue un'azione (lazy evaluation).

Per utilizzare Spark, il programmatore deve scrivere un *driver program*, che si connette a un cluster di *workers*. Il driver definisce uno o più RDD tramite trasformazioni e solo successivamente invoca su di essi azioni come quelle definite sopra.

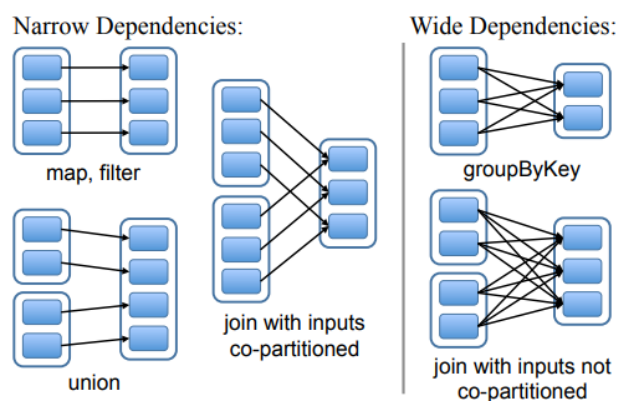
## 4.6 Rappresentazione dei RDD

L'articolo citato [7] presenta l'interfaccia utilizzata per rappresentare in Spark i RDD, la viene illustrata nella *Tabella 2*.

| Operazione                            | Significato                                                                                                              |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>partitions()</code>             | Restituisce una lista delle partizioni atomiche (Partition objects) che compongono il dataset                            |
| <code>preferredLocations(p)</code>    | Restituisce una lista dei nodi dove la partizione <i>p</i> è accessibile più velocemente grazie al principio di località |
| <code>dependencies()</code>           | Restituisce una lista delle dipendenze (i “genitori”) della partizione <i>p</i>                                          |
| <code>iterator(p, parentIters)</code> | Permette di calcolare gli elementi di una partizione <i>p</i> partendo dalle partizioni genitori                         |
| <code>partitioner()</code>            | Restituisce i metadati che specificano se il RDD è hash o range partitioned                                              |

[7] *Tabella 2. Metodi dell'interfaccia utilizzata da Spark per rappresentare i RDD*

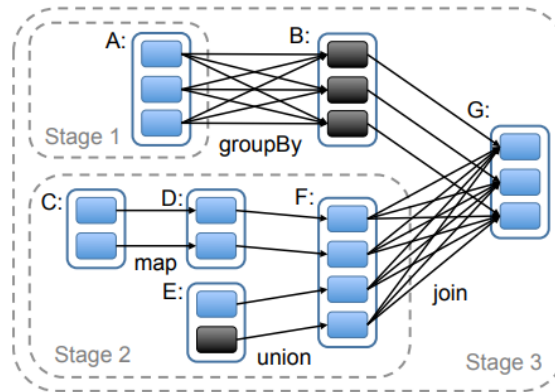
In questo contesto, le dipendenze si distinguono in due principali tipi: *narrow dependencies* (ogni partizione del genitore RDD è usata da al massimo una partizione del figlio RDD) e *wide dependencies* (più partizioni del figlio RDD possono dipendere da una singola partizione del genitore RDD). Chiaramente, l'esecuzione è meno costosa nel caso delle *narrow dependencies*, in quanto è possibile l'esecuzione in pipeline (a differenza delle *wide dependencies* dove è necessario uno shuffle).



[7] *Figura 9. Differenza tra Narrow e Wide Dependencies*

## 4.7 Job Scheduling

L'articolo citato mostra inoltre il job scheduling di Spark [7]. Quando un utente esegue un *action* (come *count* o *save*) su un RDD, Spark esamina il grafo di discendenza (lineage graph) per costruire un grafo diretto aciclico (DAG) degli stage da eseguire.



[7] Figura 10. Job scheduling di Spark

Come mostrato in *Figura 10*, ogni stage contiene il maggior numero possibile di *narrow transformations* da eseguire in un colpo solo. Ai confini degli stage si trovano invece le operazioni di shuffle necessarie per eseguire le *wide transformations*.

Per massimizzare le prestazioni, inoltre, Spark sfrutta il principio di località: se un task deve processare una partizione che si trova in uno specifico nodo, il task viene mandato ad operare direttamente su quel nodo. Oppure, se un task sta processando una partizione per cui il corrispondente RDD fornisce delle locazioni preferite, spostiamo il task su di esse.

Infine, se il task fallisce, lo si esegue su un altro nodo finché i genitori di quel dato stage sono ancora disponibili. Se non lo sono, si ricalcolano in parallelo le partizioni mancanti.

## 5. Implementazione RS collaborativo con Spark

### 5.1 Dataset Scelto

Per questa implementazione, è stato utilizzato un dataset reperito tramite GitHub, composto da tre file csv, uno per i film, uno per gli utenti e uno per i rating:

<https://github.com/lynchblue/movie-rating-dataset.git>

Tra i tre file presenti nella repository, “ratings.csv” è il dataset a cui siamo realmente interessati. Esso ha un totale di 1.172.038 ratings, ognuno dei quali è composto da 4 campi:

|                    | id                          | rate                                 | movie_id                   | timestamp             |
|--------------------|-----------------------------|--------------------------------------|----------------------------|-----------------------|
| <b>Esempio</b>     | 103007                      | 8                                    | 540542                     | 2021-03-07            |
| <b>Descrizione</b> | Identificativo per l'utente | Rating assegnato al film dall'utente | Identificativo per il film | Data della recensione |

*Tabella 3. Campi del dataset ratings.csv*

Tutti i dati sui rating e contenuti sono stati estrapolati dal sito “filmaffinity.com”, mentre quelli relativi agli utenti sono stati estrapolati da “twitter.com” (ora “X.com”).

### 5.2 Definizione della classe rating

All'interno della classe Main viene definita una classe statica annidata Rating. Essa implementa l'interfaccia Serializable, che trasforma le sue istanze in stream di byte. In questo modo, gli oggetti possono essere “spezzettati” in modo tale da parallelizzare il lavoro.

Il suo scopo è memorizzare ogni riga del dataset.

La classe rating è composta da 4 attributi, che rispecchiano i campi del file “ratings.csv”:

```
private int id;  
private int movie_id;  
private float rate;  
private String date;
```

Viene inoltre definito un costruttore vuoto, e i metodi getter e setter.

## 5.3 Fase di pre-processing

Subito dopo avere avviato la `SparkSession`, vengono letti e memorizzati sulla memoria distribuita di Spark i dati necessari per la raccomandazione dei contenuti.

In particolare, i dati vengono prima memorizzati in versione raw, e successivamente convertiti prima in oggetti della classe `rating` e infine in `DataFrame` di tipo `Dataset<Row>`, che possono essere processati adeguatamente dalle librerie di Spark.

```
String path = "data/ratings.csv";

Dataset<Row> ratingsRaw = spark.read()
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(path);

Dataset<Row> usersData = spark.read().option("header", "true")
    .option("inferSchema", "true")
    .csv("data/user_profiles.csv")
    .select("id", "twitterName");

Dataset<Row> moviesData = spark.read().option("header", "true")
    .option("inferSchema", "true")
    .csv("data/movies.csv")
    .select("id", "main_title");

Dataset<Rating> ratingsDS = ratingsRaw.as(Encoders.bean(Rating.class));
Dataset<Row> ratings = ratingsDS.toDF();
```

## 5.4 Fase di Training e Test

La fase più importante consiste nell'applicare l'algoritmo ALS che minimizza la funzione di costo in 4.2 per poi valutare le predizioni effettuate. Prima di applicare ALS bisogna però dividere il dataset in training set e test set (tramite hold-out method, si considera l'80% dei dati per la fase di training e il 20% per la fase di test).

```
Dataset<Row>[] splits = ratings.randomSplit(new double[]{0.8, 0.2});
Dataset<Row> training = splits[0];
Dataset<Row> test = splits[1];
```

A questo punto si applica l'algoritmo ALS sul training set (attualmente viene scelto 0.1 come valore per  $\lambda$ ) e si valuta il modello sul test set. Si tiene inoltre conto del problema del “cold start” menzionato in 2.3. Questo può succedere quando nuovi utenti o item non hanno uno storico di valutazioni: normalmente Spark assegna predizioni NaN, ma è possibile impostare la cold strategy a “drop” (\*), così che la validazione del modello avvenga su dati non nulli.

```
// viene applicato l'algoritmo ALS sul training set
ALS als = new ALS()
    .setMaxIter(10)          // 10 iterazioni, corrisponde a valore di default
    .setRegParam(0.1)        // valore lambda per evitare overfitting
    .setUserCol("id")
    .setItemCol("movie_id")
    .setRatingCol("rate");

ALSModel model = als.fit(training);

// valutazione del modello
model.setColdStartStrategy("drop"); // (*)
Dataset<Row> predictions = model.transform(test);

RegressionEvaluator evaluator = new RegressionEvaluator()
    .setMetricName("rmse")
    .setLabelCol("rate")
    .setPredictionCol("prediction");

double rmse = evaluator.evaluate(predictions); //precisione del modello
```

Una volta valutato il modello, sarà possibile ottenere la lista delle raccomandazioni per ogni oggetto e per ogni item semplicemente tramite i due metodi `recommendForAllUsers` e `recommendForAllItems`:

```
Dataset<Row> userRecs = model.recommendForAllUsers(5);
userRecs.show(5, false); // Mostra i primi 5 utenti

Dataset<Row> movieRecs = model.recommendForAllItems(5);
movieRecs.show(5, false);
```

## 5.5 Struttura dell'output del modello

Una volta definito il modello, è possibile visualizzarne l'output, il quale è composto dal RMSE, le raccomandazioni per gli utenti e le raccomandazioni per i film.

Nel dettaglio, la *Figura 11* mostra l'output del metodo `recommendForAllUsers(5)`, che corrisponde alle prime 5 raccomandazioni per ogni utente in ordine decrescente con il voto predetto nella forma `{movie_id, prediction}`, ovvero i 5 film le cui feature sono maggiormente allineate alle preferenze dell'utente:

```
Top 5 film per ogni utente:
+-----+
|id      |recommendations|
+-----+
|103007|[{294295, 10.763564}, {399474, 9.855198}, {809297, 9.850151}, {730528, 9.77979}, {150969, 9.599042}] |
|106161|[{877348, 8.995388}, {837955, 8.9254875}, {468373, 8.9254875}, {294295, 8.826219}, {487537, 8.814202}] |
|106757|[{253396, 9.996133}, {263596, 9.729464}, {475977, 9.707951}, {599238, 9.675076}, {294295, 9.614068}] |
|106965|[{758167, 10.699839}, {765976, 10.316593}, {284123, 10.186648}, {150969, 10.111572}, {565040, 10.095638}] |
|108525|[{294295, 10.135771}, {399474, 9.52047}, {809297, 9.238559}, {554761, 9.223235}, {930412, 9.211956}] |
+-----+
only showing top 5 rows
```

*Figura 11. Tabella di output dei primi 5 film per ogni utente*

La *Figura 12*, mostra invece il metodo `recommendForAllItems(5)`, che corrisponde alle prime 5 raccomandazioni in ordine decrescente per ogni film con il voto predetto nella forma `{id, prediction}`, ovvero i 5 utenti le cui preferenze sono maggiormente in linea con le feature del film:

```
Top 5 utenti per ogni film:
+-----+
|movie_id|recommendations|
+-----+
|100042 |[{906446, 7.526186}, {475339, 7.2390985}, {803924, 7.1046963}, {2628015, 7.07519}, {855152, 7.028011}] |
|100240 |[{906446, 8.747684}, {749914, 8.390756}, {2628015, 8.287212}, {174445, 7.860399}, {639829, 7.7250075}] |
|100320 |[{494572, 4.718997}, {136721, 4.398118}, {475339, 3.8821435}, {7141260, 3.700091}, {823876, 3.6486902}] |
|100379 |[{847702, 1.068066}, {540308, 0.9958431}, {925083, 0.9545661}, {749914, 0.9290323}, {285350, 0.9075121}] |
|100408 |[{136721, 7.6735554}, {293783, 7.517154}, {906446, 7.2328076}, {665596, 6.9403067}, {7141260, 6.5353956}] |
+-----+
only showing top 5 rows
```

*Figura 12. Tabella di output dei primi 5 utenti per ogni film*

## 5.6 Considerazioni sull'output del modello

Dando un'occhiata alla *Figura 13*, è facile notare che molti dei ratings predetti vanno oltre il massimo della scala prevista.

```
Precisione (Root Mean Squared Error) = 1.2923174187592292
Top 5 film per ogni utente:
```

| id     | recommendations                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------|
| 103007 | [{294295, 10.763564}, {399474, 9.855198}, {809297, 9.850151}, {730528, 9.77979}, {150969, 9.599042}]      |
| 106161 | [{877348, 8.995388}, {837955, 8.9254875}, {468373, 8.9254875}, {294295, 8.826219}, {487537, 8.814202}]    |
| 106757 | [{253396, 9.996133}, {263596, 9.729464}, {475977, 9.707951}, {599238, 9.675076}, {294295, 9.614068}]      |
| 106965 | [{758167, 10.699839}, {765976, 10.316593}, {284123, 10.186648}, {150969, 10.111572}, {565040, 10.095638}] |
| 108525 | [{294295, 10.135771}, {399474, 9.52047}, {809297, 9.238559}, {554761, 9.223235}, {930412, 9.211956}]      |

*Figura 13. Evidenziati i ratings “fuori scala” dei primi 5 film per ogni utente*

Questo accade perché ALS è un modello di regressione lineare basato sulla fattorizzazione di matrici, e in quanto tale non è vincolato alla scala originale (1-10). I valori che superano il 10 indicano semplicemente una forte corrispondenza tra utente e film.

Analizzando l'utente 103007 evidenziato in *Figura 14* (prima riga), si nota che il film con il massimo rating ha un punteggio di 10.76, mentre per l'utente 106161 (seconda riga) il film con il massimo rating ha un punteggio di 8.99. Ciò indica un minor grado di fiducia del modello nei confronti delle raccomandazioni per il secondo utente, probabilmente causato da un minor numero di voti storici da parte di esso.

```
Precisione (Root Mean Squared Error) = 1.2923174187592292
Top 5 film per ogni utente:
```

| id     | recommendations                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------|
| 103007 | [{294295, 10.763564}, {399474, 9.855198}, {809297, 9.850151}, {730528, 9.77979}, {150969, 9.599042}]      |
| 106161 | [{877348, 8.995388}, {837955, 8.9254875}, {468373, 8.9254875}, {294295, 8.826219}, {487537, 8.814202}]    |
| 106757 | [{253396, 9.996133}, {263596, 9.729464}, {475977, 9.707951}, {599238, 9.675076}, {294295, 9.614068}]      |
| 106965 | [{758167, 10.699839}, {765976, 10.316593}, {284123, 10.186648}, {150969, 10.111572}, {565040, 10.095638}] |
| 108525 | [{294295, 10.135771}, {399474, 9.52047}, {809297, 9.238559}, {554761, 9.223235}, {930412, 9.211956}]      |

*Figura 14. Evidenziati i ratings più alti dell'utente 103007 e 106161*

Il film 294295 appare evidenziato numerose volte in *Figura 15*: esso potrebbe essere tranquillamente considerato “blockbuster” all'interno del dataset, ma il fatto che non sia presente tra le raccomandazioni dell'utente 106965 (e anche di utenti successivi) indica una buona capacità di diversificazione da parte del modello, che porta a evitare una popolarità assoluta per tutti i profili.

```

Top 5 film per ogni utente:
+-----+-----+-----+-----+-----+-----+-----+-----+
|id      |recommendations|
+-----+-----+-----+-----+-----+-----+-----+-----+
|103007|[{294295, 10.763564}, {399474, 9.855198}, {809297, 9.850151}, {730528, 9.77979}, {150969, 9.599042}]|
|106161|[{877348, 8.995388}, {837955, 8.9254875}, {468373, 8.9254875}, {294295, 8.826219}, {487537, 8.814202}]|
|106757|[{253396, 9.996133}, {263596, 9.729464}, {475977, 9.707951}, {599238, 9.675076}, {294295, 9.614068}]|
|106965|[{758167, 10.699839}, {765976, 10.316593}, {284123, 10.186648}, {150969, 10.111572}, {565040, 10.095638}]|
|108525|[{294295, 10.135771}, {399474, 9.52047}, {809297, 9.238559}, {554761, 9.223235}, {930412, 9.211956}]|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Figura 15. Evidenziato l'ipotetico film considerato “blockbuster”, ovvero 294295

Naturalmente è possibile effettuare considerazioni analoghe alle precedenti per la tabella di output presente in *Figura 12*.

## 5.7 Considerazioni sul Root Mean Square Error e parametro $\lambda$

A scopo puramente analitico, è possibile modificare il parametro di regolarizzazione  $\lambda$  per verificare quanto anticipato in 4.2, ovvero che se  $\lambda$  è troppo piccola, verrà ignorata la regolarizzazione sommata all'errore reale (risultato del quadrato di rating reale - predizione) portando ad overfitting, mentre se è troppo grande si darà priorità a vettori troppo piccoli.

Provando ad utilizzare un valore  $\lambda = 0,0001$  è immediato notare in *Figura 16* come il modello effettui predizioni di rating estremamente alti per adattarsi strettamente ai dati di training. Aumenta inoltre il RMSE da circa 1,30 a circa 1,80:

```
Top 5 film per ogni utente:
+-----+-----+
|id      |recommendations|
+-----+-----+
|103007|[{584046, 14.366366}, {246790, 14.096751}, {474757, 13.5908985}, {809108, 12.247673}, {881751, 11.250884}]|
|106161|[{419679, 18.064571}, {947724, 15.155614}, {289761, 14.647568}, {881960, 13.854428}, {955065, 13.841909}]|
|106757|[{272074, 24.61992}, {562332, 24.067274}, {183566, 23.552517}, {303259, 22.961773}, {950666, 22.928844}]|
|106965|[{113005, 39.00963}, {645515, 34.987206}, {463629, 33.372986}, {449646, 32.810055}, {562332, 32.466152}]|
|108525|[{246790, 25.047737}, {175560, 22.763063}, {691238, 22.688698}, {886588, 21.72735}, {527868, 20.627077}]|
+-----+-----+
only showing top 5 rows
```

*Figura 16. Tabella di output dei primi 5 film per ogni utente con  $\lambda = 0,0001$*

Alternativamente, in *Figura 17* si mostra che provando ad utilizzare un valore  $\lambda = 1000$ , come immaginato si dà priorità a valori estremamente piccoli, e il RSME arriva a ben 6.34.

```
Top 5 film per ogni utente:
+-----+-----+
|id      |recommendations|
+-----+-----+
|103007|[{617694, 0.0}, {617454, 0.0}, {617311, 0.0}, {617231, 0.0}, {617221, 0.0}]|
|106161|[{617694, 0.0}, {617454, 0.0}, {617311, 0.0}, {617231, 0.0}, {617221, 0.0}]|
|106757|[{617694, 0.0}, {617454, 0.0}, {617311, 0.0}, {617231, 0.0}, {617221, 0.0}]|
|106965|[{617694, 0.0}, {617454, 0.0}, {617311, 0.0}, {617231, 0.0}, {617221, 0.0}]|
|108525|[{617694, 0.0}, {617454, 0.0}, {617311, 0.0}, {617231, 0.0}, {617221, 0.0}]|
+-----+-----+
only showing top 5 rows
```

*Figura 17. Tabella di output dei primi 5 film per ogni utente con  $\lambda = 1000$*

## 6. Conclusioni

L'analisi effettuata sui sistemi di raccomandazione ha evidenziato le differenze tra le varie categorie degli stessi, mostrandone vantaggi e svantaggi, e il focus sulla Matrix Factorization con algoritmo Alternating Least Squares e sui Resilient Distributed Datasets ha permesso di comprendere a pieno il funzionamento di Spark.

I risultati ottenuti dall'implementazione del sistema di raccomandazione hanno mostrato non solo la semplicità d'uso di Spark, ma anche la sua capacità di lavorare a grandi moli di dati grazie alle tecnologie RDD, e hanno permesso di verificare sul campo il funzionamento dell'algoritmo ALS. Il sistema di raccomandazione sviluppato ha mostrato un'ottima scalabilità su diversi profili utente: non è limitato ad evidenziare unicamente le preferenze del singolo utente, ma anche le preferenze trasversali del bacino di utenza del dataset fornito.

## 7. Bibliografia

1. Burke, R., Felfernig, A., & Göker, M. H. (2011). Recommender Systems: An Overview. *AI Magazine*, 32(3), 13-18. <https://doi.org/10.1609/aimag.v32i3.2361>
2. Dietmar Jannach and Michael Jugovac. 2019. *Measuring the Business Value of Recommender Systems*. *ACM Trans. Manage. Inf. Syst.* 10, 4, Article 16 (December 2019), 23 pages. <https://doi.org/10.1145/3370082>
3. Hodovychenko, Mykola A., and Anastasiia A. Gorbatenko. "Recommender systems: models, challenges and opportunities." *Information Technology* 6.4 (2023): 308-319.
4. A. Felfernig and R. Burke. 2008. Constraint-based recommender systems: technologies and research issues. In *Proceedings of the 10th international conference on Electronic commerce (ICEC '08)*. Association for Computing Machinery, New York, NY, USA, Article 3, 1–10. <https://doi.org/10.1145/1409540.1409544>
5. Li, S., Guo, H., Tang, X., Tang, R., Hou, L., Li, R., & Zhang, R. (2024). Embedding compression in recommender systems: A survey. *ACM Computing Surveys*, 56(5), 1-21.
6. Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," in *Computer*, vol. 42, no. 8, pp. 30-37, Aug. 2009, doi: 10.1109/MC.2009.263. keywords: {Recommender systems; Motion pictures; Filtering; Collaboration; Sea measurements; Predictive models; Genomics; Bioinformatics; Nearest neighbor searches; Computational intelligence; Netflix Prize; Matrix factorization},
7. Zaharia, Matei, et al. "Resilient distributed datasets: A {Fault-Tolerant} abstraction for {In-Memory} cluster computing." 9th USENIX symposium on networked systems design and implementation (NSDI 12). 2012.
8. *Figura 6*: <https://qdrant.tech/articles/what-are-embeddings/>